

# Generics in Java

Generics basics; bounded types; wildcards; try-with-resources

---

Rahul Guha || SME || Java FS Training || HCL

# Generics in Java

Generics in Java allow classes, interfaces, and methods to work with different data types while providing compile-time type safety. They use type parameters such as T, E, K, and V to make code reusable without requiring unnecessary type casting.

- Reduce the need for explicit type casting.
- Allow classes, interfaces, and methods to work with different reference types.
- Help detect incompatible types during compilation.

| Generic Type Syntax |             |
|---------------------|-------------|
| Type Parameter      | Meaning     |
| T                   | Type        |
| E                   | Element     |
| K                   | Key         |
| V                   | Value       |
| N                   | Number      |
| R                   | Return type |

# Why Use Generics?

- Before generics, collections such as `ArrayList` stored elements as `Object`, so retrieving an element often required explicit type casting.
- This could lead to `ClassCastException` at runtime if an object was cast to an incompatible type.
- Generics allow you to specify the type of elements a collection can store, such as `ArrayList<String>`.
- The compiler checks type safety at compile time, reducing the need for explicit casting and helping detect type-related errors earlier.
- Generics also make code easier to read by clearly indicating the type of data a collection is intended to contain.

# Types of Java Generics

## 1. Generic Class

A generic class is a class that can operate on objects of different types using a type parameter. Like C++, we use <> to specify parameter types in generic class creation. To create objects of a generic class, we use the following syntax:

// To create an instance of generic class

- `BaseType <Type> obj = new BaseType <Type>()`

## How Type Parameter T Behaves Like a Normal Type

In a generic class, the type parameter T behaves like a normal data type within the class. Once a specific type is provided while creating an object, the compiler replaces T with that type.

This means T can be used just like a regular type for:

- Declaring variables
- Method parameters
- Method return types

# Limitations of Generics in Java

## 2. Generic Method

- A generic method declares its own type parameter. The type parameter is written before the return type.

## Limitations of Generics in Java

- 1. Cannot Use Primitive Types
- Generics work with **reference types**, not primitive types.

✗ Not allowed:

```
List<int> numbers = new  
ArrayList<>();
```

☑ Use wrapper classes:

```
List<Integer> numbers = new  
ArrayList<>();
```

| Primitive | Wrapper   |
|-----------|-----------|
| int       | Integer   |
| double    | Double    |
| char      | Character |
| boolean   | Boolean   |
| long      | Long      |

# Types of Java Generics

## 2. Cannot Create Generic Objects

Directly with Type Parameter

Suppose:

```
class Box<T> {  
    T value;  
}
```

You cannot write:

```
T obj = new T(); // ❌ Error
```

**3. Cannot Create Generic Arrays** You cannot directly create an array of a type parameter.

**❌ Not allowed:**

```
class Test<T> {  
    T[] arr = new T[10]; // Error  
}
```

**4. Cannot Use instanceof with Parameterized Types**

```
if (obj instanceof ArrayList<String>) {  
    // ❌ Error  
}
```

## 2. Generic Types Differ Based on their Type Arguments

Generic types differ based on their type arguments, but this difference exists only at compile time. During compilation, Java removes generic type information through a process called type erasure, replacing type parameters with their bounds or Object. As a result, generics ensure type safety at compile time while maintaining backward compatibility at runtime.

**error:** incompatible types: Test cannot be converted to Test

//2. Generic Types Differ Based on their Type Arguments

```
class Test<T> {  
    T obj;  
  
    Test(T obj) {  
        this.obj = obj;  
    }  
  
    public void display() {  
  
        System.out.println(obj);  
    }  
}
```

```
public class a100 {  
    public static void  
    main(String[] args) {  
        Test<String> obj1 =  
        new Test<>("Rahul");  
        obj1.display();  
  
        Test<Integer> obj2 =  
        new Test<Integer>(111);  
        obj2.display();  
  
        // obj1 = obj2; // This  
        results an error  
  
    }  
}
```

# Benefits of Generics

Generics provide several benefits in Java:

**Type Safety:** Generics detect invalid type assignments at compile time, reducing the risk of runtime `ClassCastException`.

**Code Reusability:** A single generic class, method, or interface can work with different reference types.

**No Explicit Type Casting:** The compiler automatically performs the required type conversion when retrieving elements from a generic collection.

**Better Readability:** Specifying the type makes the intended data type clear and makes code easier to understand and maintain.

# Wildcards in Java

Wildcards in Java Generics allow you to work with unknown types in a flexible and type-safe way. They are represented using the ? symbol and help in writing reusable code that can handle different generic types.

- Used when the exact type of a generic parameter is not known in advance
- Commonly applied in method arguments to increase flexibility
- Helps maintain type safety while working with collections of different types

## **Types of wildcards in Java**

- In Java, wildcards are mainly classified into three types based on how they define type flexibility and restrictions.
- **1. Upper Bounded Wildcards 2. Lower Bounded Wildcards 3. Unbounded Wildcard**

# Wildcards in Java

## 1. Upper Bounded Wildcards

These wildcards can be used when you want to relax the restrictions on a variable. For example, say you want to write a method that works on `List < Integer >`, `List < Double >` and `List < Number >`, you can do this using an upper bounded wildcard.

- To declare an upper-bounded wildcard, use the wildcard character ('?'), followed by the `extends` keyword, followed by its upper bound.
- `public static void add(List<? extends Number> list)`

## 2. Lower Bounded Wildcards

- It is expressed using the wildcard character ('?'), followed by the `super` keyword, followed by its lower bound: `<? super A>`.
- Syntax: `Collectiontype <? super A>`

# Wildcards in Java

## 3. Unbounded Wildcard

This wildcard type is specified using the wildcard character (?), for example, List. This is called a list of unknown types. These are useful in the following cases -

- When writing a method that can be employed using functionality provided in Object class.
- When the code is using methods in the generic class that doesn't depend on the type parameter

# Upper Bounded vs Lower Bounded vs Unbounded Wildcards

| Feature                 | Unbounded Wildcard                        | Upper Bounded Wildcard                   | Lower Bounded Wildcard                    |
|-------------------------|-------------------------------------------|------------------------------------------|-------------------------------------------|
| <b>Syntax</b>           | <?>                                       | <? extends Type>                         | <? super Type>                            |
| <b>Meaning</b>          | Unknown type                              | Type or its subclasses                   | Type or its superclasses                  |
| <b>Example</b>          | List<?>                                   | List<? extends Number>                   | List<? super Integer>                     |
| <b>Type restriction</b> | No restriction                            | Upper limit is specified                 | Lower limit is specified                  |
| <b>Can contain</b>      | Any type                                  | Number, Integer, Double, etc.            | Integer, Number, Object                   |
| <b>Main purpose</b>     | Work with any generic type                | Read/produce values                      | Add/consume values                        |
| <b>Reading</b>          | Object                                    | Number (or bound type)                   | Object                                    |
| <b>Adding elements</b>  | ✗ Generally no                            | ✗ Generally no                           | ☑ Can add the bounded type                |
| <b>Example list</b>     | List<?>                                   | List<? extends Number>                   | List<? super Integer>                     |
| <b>Valid types</b>      | List<String>, List<Integer>, List<Double> | List<Integer>, List<Double>, List<Float> | List<Integer>, List<Number>, List<Object> |
| <b>Invalid example</b>  | —                                         | List<String> ✗                           | List<Double> ✗                            |
| <b>PECS</b>             | —                                         | <b>Producer</b> → <b>Extends</b>         | <b>Consumer</b> → <b>Super</b>            |
| <b>Best for</b>         | Unknown type                              | <b>Reading</b> data                      | <b>Writing</b> data                       |

# Comparable vs Comparator

| Feature                 | Comparable         | Comparator                 |
|-------------------------|--------------------|----------------------------|
| Package                 | java.lang          | java.util                  |
| Main method             | compareTo()        | compare()                  |
| Location of logic       | Inside class       | Outside class              |
| Sorting type            | Natural ordering   | Custom ordering            |
| Number of orderings     | Usually one        | Multiple possible          |
| Modifies original class | Yes                | No                         |
| Example                 | Sort Student by ID | Sort Student by name/marks |

# Comparable vs Comparator

| Feature                  | <code>compareTo()</code>                                     | Comparable                                                      | Comparator                                                 |
|--------------------------|--------------------------------------------------------------|-----------------------------------------------------------------|------------------------------------------------------------|
| What is it?              | A <b>method</b>                                              | An <b>interface</b>                                             | An <b>interface</b>                                        |
| Package                  | <code>java.lang</code>                                       | <code>java.lang</code>                                          | <code>java.util</code>                                     |
| Main method              | <code>compareTo()</code>                                     | <code>compareTo()</code>                                        | <code>compare()</code>                                     |
| Purpose                  | Performs comparison                                          | Defines <b>natural/default ordering</b>                         | Defines <b>custom ordering</b>                             |
| Where implemented?       | Inside the class                                             | The class being sorted                                          | Usually a separate class/lambda                            |
| Modifies original class? | —                                                            | <b>Yes</b>                                                      | <b>No</b>                                                  |
| Number of sorting rules  | Usually one natural order                                    | Usually one natural order                                       | Can have <b>multiple</b>                                   |
| Example                  | <code>s1.compareTo(s2)</code>                                | <code>class Student implements Comparable&lt;Student&gt;</code> | <code>Comparator&lt;Student&gt;</code>                     |
| Used with                | <code>Collections.sort()</code> / <code>Arrays.sort()</code> | <code>Collections.sort()</code> / <code>Arrays.sort()</code>    | <code>Collections.sort()</code> / <code>List.sort()</code> |
| Best use                 | Compare two objects according to natural order               | Define object's default sorting                                 | Different/custom sorting requirements                      |